Debug of SV and UVM

UVM Debugging

Built-In Debug

While simulators should provide (and do provide in the case of QuestaSim [1]) useful debug capabilities to help diagnose issues when they happen in a UVM testbench, it also is useful to know the built-in debug features that come with the UVM library. UVM provides a vast code base with ample opportunities for subtle issues. This article will explore the functions and plusargs that are available out of the box with UVM to help with debug.

Note that the majority of the features described in this article are not documented in the the IEEE 1800.2 LRM, however they are supported as debug features of the Accellera implementation of the UVM 1800.2 class library. The features described are available in earlier versions of the UVM (1.1a through to 1.2).

Configuration Debug Features

The UVM Resource Database is used to pass configuration information from a test down into a testbench. It is one of the ways that the test controls "what" is going to happen in the testbench leaving the testbench to decide "how" it is going to happen. This mechanism is very powerful, but it does rely on string matching to function correctly. To that end, UVM provides a few capabilities to help ensure those strings match up.

UVM Command Line Processor

The UVM Command Line Processor can be used to turn on trace messages which then print out UVM_INFO messages showing when information is placed into the resource database (a set) or pulled out of the resource database (a get). The command line processor provides two plusargs: +UVM_RESOURCE_DB_TRACE and +UVM_CONFIG_DB_TRACE. +UVM_RESOURCE_DB_TRACE is used when the uvm_resource_db API is used in the SystemVerilog source code. +UVM_CONFIG_DB_TRACE is used when the uvm_config_db API is used. When turned on, the output will look something like this:

```
UVM_INFO ../uvm-1.2/src/base/uvm_resource_db.svh(129) @ 0 ns: reporter [CFGDB/
SET] Configuration 'uvm_test_top.mem_interface' (type virtual mem_interface)
set by  = (virtual mem_interface) ?

UVM_INFO ../uvm-1.2/src/base/uvm_resource_db.svh(129) @ 0 ns: reporter [CFGDB/
GET] Configuration 'uvm_test_top.mem_interface' (type virtual mem_interface)
read by uvm_test_top = (virtual mem_interface) ?
```

UVM Component

Additionally each class derived from uvm_component which would include drivers, monitors, agents, environments, etc. comes with some built-in functions to help with configuration debug.

Function	Prototype	Description

print_config	`function void print_config( bit recurse = 0,`
	
	`bit audit = 0 )`
		Print_config_settings prints all configuration information for this component, as set by previous calls to set_config_* and exports to the resources pool. The settings are printing in the order of their precedence.
		
		If *recurse* is set, then configuration information for all children and below are printed as well.
		
		if *audit* is set then the audit trail for each resource is printed along with the resource name and value

print_config_with_audit	`function void print_config_with_audit( bit recurse = 0 )`	Operates the same as print_config except that the audit bit is forced to 1. This interface makes user code a bit more readable as it avoids multiple arbitrary bit settings in the argument list. If *recurse* is set, then configuration information for all children and below are printed as well.

These functions could be called from the build_phase, connect_phase or most likely the end_of_elaboration_phase. These functions can also be used without modifying the source code by using the Questa specific "call" TCL command. The "call" command allows for SV functions be be called from the TCL command line. An example of using the call command to print the config would look like this:

```
call [examine -handle
{sim:/uvm_pkg::uvm_top.top_levels[0].super.m_env.m_mem_agent.m_monitor}].print_config
```

Resource Database Dump

Another option for exploring what is currently in the resource database is the dump() command. When used, it will print out what is currently in the resource database along with the types and paths that are stored for each item. To use this facility, uvm_config_db #(<type>)::dump() needs to be added to your source code. "<type>" can be anything including int, bit, or some user defined type. When this is added, output similiar to the following will be seen.

```
# === resource pool ===
# mem_config [/^uvm_test_top\..*mem_agent.*$/] : (class
mem_agent_pkg::mem_config)
# ----------------------------------
# Name       Type        Size  Value
# ----------------------------------
# m_mem_cfg  mem_config  -     @552
# ----------------------------------
#
# -
# mem_interface [/^uvm_test_top$/] : (virtual mem_interface) ?
# -
# mem_intf_mon_mp [/^uvm_test_top$/] : (virtual
mem_interface.monitor_mp) ?
# -
# === end of resource pool ===
```

This function could be called from the build_phase, connect_phase or most likely the end_of_elaboration_phase.

Factory Debug Features

The UVM Factory is used to create objects in UVM. It also allows the test another mechanism for controlling "what" is going to happen in a testbench by the use of factory overrides. The Factory is created automatically as a singleton object with a handle called "factory" living in the uvm_pkg. This means that functions can be called on this factory singleton to help understand who is registered with the Factory, which overrides the Factory currently is using and to test out what objects would be returned by the factory for a given type. There are three functions which provide this information.

Function	Prototype	Description
print	`function void print ( int all_types = 1 )`	Prints the state of the uvm_factory, including registered types, instance overrides, and type overrides. When *all_types* is 0, only type and instance overrides are displayed. When *all_types* is 1 (default), all registered user-defined types are printed as well, provided they have names associated with them. When *all_types* is 2, the UVM types (prefixed with uvm_) are included in the list of registered types.

Function	Prototype	Description
debug_create_by_type	`function void debug_create_by_type ( uvm_object_wrapper requested_type, string parent_inst_path = "", string name = "" )`	These methods perform the same search algorithm as the create_* methods, but they do not create new objects. Instead, they provide detailed information about what type of object it would return, listing each override that was applied to arrive at the result. When requesting by type, the *requested_type* is a handle to the type's proxy object. Preregistration is not required.
debug_create_by_name	`function void debug_create_by_name ( string requested_type_name, string parent_inst_path = "", string name = "" )`	When requesting by name, the *request_type_name* is a string representing the requested type, which must have been registered with the factory with that name prior to the request. If the factory does not recognize the requested_type_name, an error is produced and a null handle returned. If the optional *parent_inst_path* is provided, then the concatenation, {parent_inst_path, ".",~name~}, forms an instance path (context) that is used to search for an instance override. The *parent_inst_path* is typically obtained by calling the uvm_component::get_full_name on the parent.

The most commonly used of these function is print. That is used in the form factory.print() which returns information similar to the following which includes which classes are registered with the factory and any overrides which are registered with the factory.

```
#### Factory Configuration (*)
#
# No instance overrides are registered with this factory
#
# Type Overrides:
#
#   Requested Type  Override Type
#   --------------  -------------
#   mem seq base    mem seq1
#
# All types registered with the factory: 72 total
# (types without type names will not be printed)
#
```

```
#   Type Name
#   ---------
#   analysis_group
#   coverage
#   directed_test1
#   environment
#   mem_agent
#   mem_config
#   mem_driver
#   mem_item
#   mem_monitor
#   mem_seq1
#   mem_seq2
#   mem_seq_base
#   mem_seq_lib
#   mem_trans_recorder
#   questa_uvm_recorder
#   scoreboard
#   test1
#   test_base
#   test_predictor
#   test_seq_lib
#   threaded_scoreboard
# (*) Types with no associated type name will be printed as <unknown>
#
####
```

These functions could be called from the build_phase, connect_phase or most likely the end_of_elaboration_phase. Factory.print() could also be called using the Questa "call" command like this:

```
uvm printfactory
```

NOTE: In UVM1.2, the factory cannot be referenced directly in the uvm_pkg. Instead, it must be accessed via

```
uvm_factory::get();
```

Phasing Debug Features

Phasing in UVM can be complicated with multiple phases running in parallel. To help understand when a phase starts and ends, the UVM Command Line Processor can be used to enable a trace with the +UVM_PHASE_TRACE plusarg. When this plusarg is added to the simulator command line, it results in output similiar to this:

```
# UVM_INFO ../uvm-1.1a/src/base/uvm_phase.svh(1364) @ 0 ns: reporter [PH/TRC/
SCHEDULED] Phase 'common.run' (id=93) Scheduled from phase
common.start_of_simulation

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1114) @ 0 ns: reporter [PH/TRC/STRT]
Phase 'common.run' (id=93) Starting phase
```

```
# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1364) @ 0 ns: reporter [PH/TRC/SCHEDULED] Phase 'uvm' (id=142) Scheduled from phase common.start_of_simulation

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1114) @ 0 ns: reporter [PH/TRC/STRT] Phase 'uvm' (id=142) Starting phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1342) @ 0 ns: reporter [PH/TRC/DONE] Phase 'uvm' (id=142) Completed phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1364) @ 0 ns: reporter [PH/TRC/SCHEDULED] Phase 'uvm.uvm_sched' (id=154) Scheduled from phase uvm

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1114) @ 0 ns: reporter [PH/TRC/STRT] Phase 'uvm.uvm_sched' (id=154) Starting phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1342) @ 0 ns: reporter [PH/TRC/DONE] Phase 'uvm.uvm_sched' (id=154) Completed phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1364) @ 0 ns: reporter [PH/TRC/SCHEDULED] Phase 'uvm.uvm_sched.pre_reset' (id=172) Scheduled from phase uvm.uvm_sched

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1114) @ 0 ns: reporter [PH/TRC/STRT] Phase 'uvm.uvm_sched.pre_reset' (id=172) Starting phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1191) @ 0 ns: reporter [PH/TRC/SKIP] Phase 'uvm.uvm_sched.pre_reset' (id=172) No objections raised, skipping phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1191) @ 0 ns: reporter [PH/TRC/SKIP] Phase 'common.run' (id=93) No objections raised, skipping phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1342) @ 0 ns: reporter [PH/TRC/DONE] Phase 'uvm.uvm_sched.pre_reset' (id=172) Completed phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1364) @ 0 ns: reporter [PH/TRC/SCHEDULED] Phase 'uvm.uvm_sched.reset' (id=184) Scheduled from phase uvm.uvm_sched.pre_reset

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1114) @ 0 ns: reporter [PH/TRC/STRT] Phase 'uvm.uvm_sched.reset' (id=184) Starting phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1342) @ 80 ns: reporter [PH/TRC/DONE] Phase 'uvm.uvm_sched.reset' (id=184) Completed phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1364) @ 80 ns: reporter [PH/TRC/SCHEDULED] Phase 'uvm.uvm_sched.post_reset' (id=196) Scheduled from phase uvm.uvm_sched.reset

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1114) @ 80 ns: reporter [PH/TRC/STRT] Phase 'uvm.uvm_sched.post_reset' (id=196) Starting phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1191) @ 80 ns: reporter [PH/TRC/SKIP] Phase 'uvm.uvm_sched.post_reset' (id=196) No objections raised, skipping phase

# UVM_INFO ../uvm-1.2/src/base/uvm_phase.svh(1342) @ 80 ns: reporter [PH/TRC/DONE] Phase 'uvm.uvm_sched.post_reset' (id=196) Completed phase
```

Objection Debug Features

Objections are used to control when a time consuming phase is going to end. Understanding when an objection is raised or lowered can be a difficult proposition especially since all of the raise objection calls must be matched with an equal number of drop objection calls. The UVM Command Line Processor can be used to enable a trace with the +UVM_OBJECTION_TRACE plusarg. When this plusarg is added to the simulator command line, it results in output similar to this:

```
# UVM_INFO @ 0 ns: reset_objection [OBJTN_TRC] Object uvm_test_top raised 1 objection(s) (Raising Reset Objection): count=1 total=1

# UVM_INFO @ 0 ns: reset_objection [OBJTN_TRC] Object uvm_top added 1 objection(s) to its total (raised from source object uvm_test_top, Raising Reset Objection): count=0 total=1

# UVM_INFO @ 80 ns: reset_objection [OBJTN_TRC] Object uvm_test_top dropped 1 objection(s) (Dropping Reset Objection): count=0 total=0

# UVM_INFO @ 80 ns: reset_objection [OBJTN_TRC] Object uvm_test_top all_dropped 1 objection(s) (Dropping Reset Objection): count=0 total=0

# UVM_INFO @ 80 ns: reset_objection [OBJTN_TRC] Object uvm_top subtracted 1 objection(s) from its total (dropped from source object uvm_test_top, Dropping Reset Objection): count=0 total=0

# UVM_INFO @ 80 ns: reset_objection [OBJTN_TRC] Object uvm_top subtracted 1 objection(s) from its total (all_dropped from source object uvm_test_top, Dropping Reset Objection): count=0 total=0

# UVM_INFO @ 80 ns: main_objection [OBJTN_TRC] Object uvm_test_top raised 1 objection(s) (Raising Main Objection): count=1 total=1

# UVM_INFO @ 80 ns: main_objection [OBJTN_TRC] Object uvm_top added 1 objection(s) to its total (raised from source object uvm_test_top, Raising Main Objection): count=0 total=1

# UVM_INFO @ 5350 ns: main_objection [OBJTN_TRC] Object uvm_test_top dropped 1 objection(s) (Dropping Main Objection): count=0 total=0

# UVM_INFO @ 5350 ns: main_objection [OBJTN_TRC] Object uvm_test_top all_dropped 1 objection(s) (Dropping Main Objection): count=0 total=0

# UVM_INFO @ 5350 ns: main_objection [OBJTN_TRC] Object uvm_top subtracted 1 objection(s) from its total (dropped from source object uvm_test_top, Dropping Main Objection): count=0 total=0

# UVM_INFO @ 5350 ns: main_objection [OBJTN_TRC] Object uvm_top subtracted 1 objection(s) from its total (all_dropped from source object uvm_test_top, Dropping Main  Objection): count=0 total=0
```

TLM Port Debug Features

Components in UVM are connected together via TLM ports, exports and imps. UVM provides two functions which can be called on a port, export or imp to help understand which objects are connected together. Those two functions are get_connected_to() (usually used with ports) and get_provided_to() (usually used with imps or exports).

Note: The get_connected_to() and get_provided_to() functions were renamed in the IEEE 1800.2. In previous versions of the UVM, the two calls were named debug_connected_to() and debug_provided_to(), their functionality is as described here.

Function	Prototype	Description

print_config	`function void print_config( bit recurse = 0,`
	
	`bit audit = 0 )`
		Print_config_settings prints all configuration information for this component, as set by previous calls to set_config_* and exports to the resources pool. The settings are printing in the order of their precedence.
		
		If *recurse* is set, then configuration information for all children and below are printed as well.
		
		if *audit* is set then the audit trail for each resource is printed along with the resource name and value

print_config_with_audit	`function void print_config_with_audit( bit recurse = 0 )`	Operates the same as print_config except that the audit bit is forced to 1. This interface makes user code a bit more readable as it avoids multiple arbitrary bit settings in the argument list. If *recurse* is set, then configuration information for all children and below are printed as well.

These functions should be called from the end_of_elaboration_phase as that is when all connections will have been completed by then.

The get_connected_to() function could also be used in a call command like this:

```
call [examine -handle
{sim:/
uvm_pkg::uvm_top.top_levels[0].super.m_env.m_mem_agent.m_monitor.result_from_m
onitor_ap}].get_connected_to
```

When get_connected_to() is used on a port, output similar to the following should be seen:

```
# UVM_INFO @ 0 ns:
uvm_test_top.m_env.m_mem_agent.m_monitor.result_from_monitor_ap
[get_connected_to] This port's fanout network:
#
#   uvm_test_top.m_env.m_mem_agent.m_monitor.result_from_monitor_ap
(uvm_analysis_port)
#     |
```

```
#     |_uvm_test_top.m_env.m_mem_agent.m_mon_out_ap (uvm_analysis_port)
#         |
#         | uvm test top.m env.m analysis.analysis export
(uvm analysis export)
#         |
#         | uvm test top.m env.m analysis.coverage h.analysis imp
(uvm_analysis_imp)
#             |
#
|_uvm_test_top.m_env.m_analysis.test_predictor_h.analysis_imp
(uvm_analysis_imp)
#
#   Resolved implementation list:
#   0: uvm_test_top.m_env.m_analysis.coverage_h.analysis_imp
(uvm_analysis_imp)
#   1: uvm_test_top.m_env.m_analysis.test_predictor_h.analysis_imp
(uvm analysis imp)
```

The get_provided_to() function could also be used in a call command like this:

```
call [examine -handle
{sim:/uvm_pkg::uvm_top.top_levels[0].super.m_env.m_analysis.coverage_h.super.analysis_export}].get_provided_to
```

When get_provided_to() is used on an export or an imp, output similar to the following should be seen:

```
# UVM_INFO @ 0 ns:
uvm_test_top.m_env.m_analysis.coverage_h.analysis_imp [get_provided_to]
 This port's fanin network:
#
#   uvm_test_top.m_env.m_analysis.coverage_h.analysis_imp
(uvm_analysis_imp)
#     |
#     |_uvm_test_top.m_env.m_analysis.analysis_export (uvm_analysis_export)
#         |
#         |_uvm_test_top.m_env.m_mem_agent.m_mon_out_ap(uvm_analysis_port)
#             |
#
|_uvm_test_top.m_env.m_mem_agent.m_monitor.result_from_monitor_ap
(uvm_analysis_port)
```

Callback Debug Features

Callbacks allow for functions and tasks to be called on an external object from a standard object.

Display Function

If callbacks are used in a testbench, UVM can print out all of the currently registered callbacks using the display function.

Function	Prototype	Description
display	`static function void display( T obj = null )`	This function displays callback information for obj. If obj is null, then it displays callback information for all objects of type T, including typewide callbacks.

This function should be called from the end_of_elaboration_phase. As an example, to display all the callbacks registered with the uvm_reg class, the SystemVerilog code would have the following line added:

```
uvm_reg_cb::display();
```

This results in output that looks like the following:

```
# Registered callbacks for all instances of uvm_reg
# ---------------------------------------------------------------
# status_reg_h_cb   on dut_rm.status_reg_h  ON
# RegA_h_cb         on dut_rm.RegA_h        ON
# RegB_h_cb         on dut_rm.RegB_h        ON
```

Compile Time Option

Additionally, if the uvm_pkg is being compiled manually, another option exists which will enable tracing of callbacks. When compiling the uvm_pkg, +define+UVM_CB_TRACE_ON can be added which turns on output similiar to the following:

```
UVM_INFO ../uvm-1.1a/src/base/uvm_callback.svh(1142) @ 120 ns: reporter
 [UVMCB_TRC] Callback mode for status_reg_h_cb is  ENABLED : callback
status_reg_h_cb (uvm_callback@837)
```

This is displayed when a callback is accessed in simulation.

General Debug Features

Some other functions and techniques are available as well.

Component Hierarchy

To print out the current component hierarchy of a testbench, the print_topology() function can be called.

(Note: print_topology() is documented in the IEEE 1800.2 LRM)

Function	Prototype	Description
print_topology	`function void print_topology ( uvm_printer printer = null )`	Print the verification environment's component topology. The printer is a uvm_printer object that controls the format of the topology printout; a null printer prints with the default output.

Generally, this function would be called from the end_of_elaboration phase. There is also a bit called enable_print_topology which defaults to 0 and is a data member of the uvm_root class. When this bit is set to a 1, the entire testbench topology is printed just after completion of the end_of_elaboration phase. Since the uvm_pkg::uvm_top handle points to a singleton of the uvm_root class, this bit could be set by adding

```
uvm_top.enable_print_topology = 1;
```

to the testbench code. The print_topology functon can also be called from the Questa command line using the "call" command like this:

```
call /uvm_pkg::uvm_top.print_topology
```

Verbosity Controls

The messaging system in UVM also provides a robust way of controlling verbosity. Useful UVM_INFO messages could be sprinkled throughout the testbench with their verbosity set to UVM_HIGH which would cause them to normally not be printed. When a user would like to have these messages be displayed, then the UVM Command Line Processor comes into play. It has a global verbosity setting which is available using the +UVM_VERBOSITY plusarg or individual components can have thier verbosity settings altered by using the +uvm_set_verbosity plusarg.

PlusArg	Description
+UVM_VERBOSITY=<uvm_verbosity>	This will set the default verbosity of all the components created in the UVM testbench to the specified verbosity.
+uvm_set_verbosity=<component>,<id>,<verbosity>,<phase>	This will set the verbosity of a specific component for a given UVM phase
+uvm_set_verbosity=<component>,<id>,<verbosity>,time,<time>	This will set the verbosity of a specific component from a specific time in the simulation.

DVCon Papers

Papers have also been written about how to debug class based environments. One paper which was the runner up for the DVCon 2012 best paper award is Better Living Through Better Class-Based SystemVerilog Debug [2]. This paper contains several other useful techniques which can be employed to help with UVM debug.

References

[1] http://www.mentor.com/products/fv/questa/
[2] http://events.dvcon.org/2012/proceedings/papers/12_3.pdf

Using Verbosity for Debug

Introduction

UVM provides a built-in mechanism to control how many messages are printed in a UVM based testbench. This mechanism is based on comparing integer values specified when creating a debug message using either the uvm_report_info() function or the `uvm_info() macro.

Verbosity Levels

UVM defines an enumerated value (uvm_verbosity) which provides several predefined levels. These levels are used in two places. The first place is when writing the message in the code. The second place is the setting that every uvm_component possesses to determine a message should be displayed or filtered.

Message Verbosity Setting	Component Verbosity Setting				
	UVM_NONE	UVM_LOW	UVM_MEDIUM	UVM_HIGH	UVM_FULL
UVM_NONE	Displayed	Displayed	Displayed	Displayed	Displayed
UVM_LOW	Filtered	Displayed	Displayed	Displayed	Displayed
UVM_MEDIUM	Filtered	Filtered	Displayed	Displayed	Displayed
UVM_HIGH	Filtered	Filtered	Filtered	Displayed	Displayed
UVM_FULL	Filtered	Filtered	Filtered	Filtered	Displayed

The default verbosity level out of the box is UVM_MEDIUM which means that every message with verbosity level of UVM_MEDIUM, UVM_LOW or UVM_NONE will be displayed. As you set a components verbosity level to a higher setting, it will expose more detail (become more verbose) in the transcript. This is a bit counter-intuitive for a lot of people as they expect messages with a "higher" verbosity setting to always be printed. In UVM, messages with a "lower" verbosity setting will be printed before messages with a higher setting. In fact, a message with a UVM_NONE setting will always be printed.

Components contain a verbosity setting which is compared against to determine if a message will be displayed or not. There is also the root uvm_top component whose verbosity setting is used when messages are composed in other objects such as configuration objects which are not derived from uvm_component or if a message is composed in a module.

Sequences have their own set of rules for determining which component to use for accessing the current verbosity setting. If a sequence is running on a sequencer (sequence.start(sequencer)), then the sequence uses the verbosity setting of the sequencer it is running on. If a sequence is started with a null sequencer handle (for example a virtual sequence), then uvm_top's verbosity setting is used. Sequence Items follow the similar rules. When a sequence item is created in a sequence, it will use the verbosity setting of the sequencer the sequence is running on if the sequencer handle is not null. Otherwise, the sequence item defaults to using uvm_top's verbosity setting.

Sequences and sequence items behave differently because uvm_report_info/warning/error/fatal() functions are defined in the uvm_sequence_item class which uvm_sequence inherits from. Inside these functions, there is a check to see if the sequencer handle is null or not. If the sequencer handle is not null, it's verbosity setting is used. If it is null, then uvm_top's verbosity setting is used. This check is performed the first time a uvm_report_info/warning/error/fatal() function is called.

Note: If a sequence is started on one sequencer, prints out a message, finishes and then is started on another sequencer, the original sequencer's verbosity setting will be used. This is an issue in UVM which will be fixed in a future release.

Usage

To take advantage of the different verbosity levels, a testbench needs to use the `uvm_info macro (which calls the uvm_report_info() function). The third argument is what is used for controlling verbosity. For example:

```
task run_phase(uvm_phase phase);
   `uvm_info({get_type_name(),"::run_phase"}, "Starting run_phase",
UVM_HIGH)

   ...

   `uvm_info({get_type_name(),"::run_phase"}, "Checkpoint 1 of
run_phase", UVM_MEDIUM)

   ...

   `uvm_info({get_type_name(),"::run_phase"}, "Checkpoint 2 of
run_phase", UVM_LOW)

   ...

   `uvm_info({get_type_name(),"::run_phase"}, "Ending run_phase",
UVM_HIGH)

endtask : run_phase
```

Running this code with default settings would only result in the Checkpoint 1 and Checkpoint 2 messages being printed.

Note: only "info" messages can be masked using verbosity settings. "warning", "error" and "fatal" messages will always be printed.

An additional way to use verbosity settings is to explicitly check what the current verbosity setting is. The uvm_report_enabled() function will return a 0 or a 1 to inform the user if the component is currently configured to print messages at the verbosity level passed in. Code can then explicitly check the current verbosity setting before calling functions which display information (item.print(), etc.) which don't take into account verbosity settings.

```
task run_phase(uvm_phase phase);
   ...

   if (uvm_report_enabled(UVM_HIGH))
      item.print();

   ...
endtask : run_phase
```

Here we are checking if UVM_HIGH messages should be printed. If they should be printed, then we will print out the item.

Message ID Guidance

Every message that is printed contains an ID which is associated with the message. The ID field can be used for any kind of user-specified identification or filtering, outside the simulation. Within the simulation, the ID can be used alongside the severity, verbosity attributes to configure any report's actions, output file(s), or make other decisions to modify or suppress the report. This capability is enhanced further in UVM with the UvmMessageCatching API.

Since verbosity can get down to an ID level of granularity, it is recommended to follow the pattern of concatenating get_type_name() together with a secondary string. The static function get_type_name() (created by `uvm_component/object_utils() macro) will inform the user which class the string is coming from. The secondary string can then be used to identify the function/task where the message was composed or for further granularity as needed.

Verbosity Controls

With messages in place which will use verbosity settings, the functions and plusargs which allow controlling the current verbosity setting need to be discussed. These functions and plusargs can be divided into global controls and more fine grain controls.

Global Controls

To globally set a verbosity level for a testbench from the test on down, the set_report_verbosity_level_hier() function would be used.

```
// Function: set_report_verbosity_level_hier
  //
  // This method recursively sets the maximum verbosity level for
reports for
  // this component and all those below it. Any report from this
component
  // subtree whose verbosity exceeds this maximum will be ignored.
  //
```

```
  // See <uvm_report_handler> for a list of predefined message verbosity levels
  // and their meaning.

    extern function void set_report_verbosity_level_hier (int
verbosity);
```

This function when called from a testbench module will globally set the verbosity setting for the entire UVM testbench. This would look like this:

```
module testbench;
  import uvm_pkg::*;
  import uvm_tests_pkg::*;

  //Dut instantiation, etc.

  initial begin
    // Other TB initialization code such as config_db calls to pass
virtual
    // interface handles to the testbench

    //Turn on extra debug messages
    uvm_top.set_report_verbosity_level_hier(UVM_HIGH);

    //Run the test (UVM_TESTNAME will override "test1" if set on cmd
line)
    run_test("test1");
  end

endmodule : testbench
```

An even easier way which doesn't require recompilation is to use the UVM Command Line Processor to process the +UVM_VERBOSITY plusarg. This would look like this:

```
vsim testbench +UVM_VERBOSITY=UVM_HIGH
```

Fine Grain Controls

In addition to the global controls, UVM allows for individual setting of verbosity levels at a component level and even at an ID level of granularity.

To set an individual components verbosity level, the set_report_verbosity_level() function can be used.

```
// Function: set_report_verbosity_level
  //
  // This method sets the maximum verbosity level for reports for this
component.
  // Any report from this component whose verbosity exceeds this
maximum will
  // be ignored.

  function void set_report_verbosity_level (int verbosity_level);
```

There is also a hierarchical version of this function which will set a component's and its children's verbosity level.

```
function void set_report_verbosity_level_hier ( int verbosity );
```

The UVM Command Line Processor also provides for more fine grain control by using the +uvm_set_verbosity plusarg. This plusarg allows for component level and/or ID level verbosity control.

There are several other functions which can be used to change a specific ID's verbosity level, change actions for specific severities, etc. These methods are documented in the UVM Reference guide.

Performance Considerations

This article mentions both the uvm_report_info() function and the `uvm_info() macro. Mentor recommends using the `uvm_info() macro for dealing with reporting for a couple reasons (detailed in the Macro Cost Benefit DVCon paper). The most important of those reasons is that the `uvm_info() macro can significantly speed up simulations when a verbosity setting is such that messages won't be printed. The reason for the speed up is the macro checks the verbosity setting before doing any string processing.

Looking at a simple `uvm_info() statement:

```
`uvm_info("Message_ID", $sformatf("%s, data[%0d] = 0x%08x", name, ii,
data[ii]), UVM_HIGH)

there is a fairly complex $sformat statement contained within the
`uvm_info() statement.  The $sformat() statement would take enough
simulation time to be noticable when called repeatedly.  With a
verbosity setting of UVM_HIGH, this $sformat() message won't be printed
 most of the time.  Because of the macro usage, the processing time
required to generate the resultant string from processing the
$sformat() message won't be wasted as well.
```

The `uvm_info() macro under the hood is using the uvm_report_object::uvm_report_enabled() API to perform an inexpensive check of the verbosity setting to determine if a message will be printed or not before ultimately calling uvm_report_info() with the same arguments as the macro if the check returns a positive result.

```
if (uvm_report_enabled(UVM_HIGH, UVM_INFO, "Message_ID"))
  uvm_report_info("Message_ID", $sformatf("%s, data[%0d] = 0x%08x",
name, ii, data[ii]), UVM_HIGH);
```

If uvm_report_info() was used directly instead

```
uvm_report_info("Message_ID", $sformatf("%s, data[%0d] = 0x%08x", name,
 ii, data[ii]), UVM_HIGH);
```

the $sformat() string would have be processed before entering the uvm_report_info() function even through ultimately that string will not be printed due to the UVM_HIGH verbosity setting.

Command-Line Processor

Introduction

The UVM command line processor is used to interact with plusargs. Several plusargs are pre-defined and part of the UVM standard. The predefined plusargs allow for modification of built-in UVM variables including verbosity settings, setting integers and strings in the resource database, and controlling tracing for phases and resource database accesses. The command line processor also eases interacting with user defined plusargs.

The IEEE 1800.2 LRM defines a sub-set of the built-in UVM plusargs, the Accellera UVM 1800.2 implementation supports all the plusargs, most importantly the ones that enable debug features in the library.

In general, plusargs should be used only to temporarily change what is going to happen in a testbench. For example, to enable additional debug information when something goes wrong or to control transaction recording when running in GUI mode.

Built-In UVM Plusargs

There are two classes of built-in UVM plusargs. The first class are plusargs which can only be set once on the command line. The second class are plusargs which can be set multiple times.

Single Use Plusargs

Plusargs documented in the IEEE 1800.2 LRM:

Plusarg Name	Description	Example Usage
+UVM_TESTNAME	+UVM_TESTNAME=<class name> allows the user to specify which uvm_test (or uvm_component) should be created via the factory and cycled through the UVM phases. If multiple of these settings are provided, the first occurrence is used and a warning is issued for subsequent settings.	vsim testbench +UVM_TESTNAME="block_test1"
+UVM_VERBOSITY	+UVM_VERBOSITY=<verbosity> allows the user to specify the initial verbosity for all components. The uvm_verbosity enum values (UVM_LOW, UVM_MEDIUM, UVM_HIGH, etc.) and integer values are accepted as arguments. If multiple of these settings are provided, the first occurrence is used and a warning is issued for subsequent settings.	vsim testbench +UVM_VERBOSITY=UVM_HIGH
+UVM_TIMEOUT	+UVM_TIMEOUT=<timeout>,<overridable> allows users to change the global timeout of the UVM framework. The timeout value is specified as an interger number of ns. Time specifiers such as ms or us can not be used currently. The <overridable> argument ('YES' or 'NO') specifies whether user code can subsequently change this value. If set to 'NO' and the user code tries to change the global timeout value, a warning message will be generated. The default value for <overridable> is 'YES'.	vsim testbench +UVM_TIMEOUT=1000000,NO

+UVM_MAX_QUIT_COUNT	+UVM_MAX_QUIT_COUNT=<count>,<overridable> allows users to change max quit count for the report server. The <overridable> argument ('YES' or 'NO') specifies whether user code can subsequently change this value. If set to 'NO' and the user code tries to change the max quit count value, an warning message will be generated. The default value for <overridable> is 'YES'.	vsim testbench +UVM_MAX_QUIT_COUNT=5,NO

Plusargs not documented in the IEEE 1800.2 LRM, but available in the Accellera UVM implementations

Plusarg Name	Description	Example Usage
+UVM_TIMEOUT	+UVM_TIMEOUT=<timeout>,<overridable> allows users to change the global timeout of the UVM framework. The timeout value is specified as an interger number of ns. Time specifiers such as ms or us can not be used currently. The <overridable> argument ('YES' or 'NO') specifies whether user code can subsequently change this value. If set to 'NO' and the user code tries to change the global timeout value, a warning message will be generated. The default value for <overridable> is 'YES'.	vsim testbench +UVM_TIMEOUT=1000000,NO
+UVM_PHASE_TRACE	+UVM_PHASE_TRACE turns on tracing of phase executions. Users simply need to put the argument on the command line.	vsim testbench +UVM_PHASE_TRACE
+UVM_OBJECTION_TRACE	+UVM_OBJECTION_TRACE turns on tracing of objection activity. If a description was supplied when raising or dropping an objection, it will be displayed when this plusarg is set. Users simply need to put the argument on the command line.	vsim testbench +UVM_OBJECTION_TRACE
+UVM_RESOURCE_DB_TRACE	+UVM_RESOURCE_DB_TRACE turns on tracing of resource DB access. Users simply need to put the argument on the command line.	vsim testbench +UVM_RESOURCE_DB_TRACE
+UVM_CONFIG_DB_TRACE	+UVM_CONFIG_DB_TRACE turns on tracing of configuration DB access. Users simply need to put the argument on the command line.	vsim testbench +UVM_CONFIG_DB_TRACE

Multiple Use Plusargs

All of these plusargs are documented in the IEEE 1800.2 LRM

Plusarg Name	Description	Example Usage
+uvm_set_verbosity	+uvm_set_verbosity=<comp>,<id>,<verbosity>,<phase> and +uvm_set_verbosity=<comp>,<id>,<verbosity>,time,<time> allow the users to manipulate the verbosity of specific components at specific phases (and times during the "run" phases) of the simulation. The id argument can be either _ALL_ for all IDs or a specific message id. Wildcarding is not supported for id due to performance concerns. Settings for non-"run" phases are executed in order of occurrence on the command line. Settings for "run" phases (times) are sorted by time and then executed in order of occurrence for settings of the same time.	`vsim testbench \ +uvm_set_verbosity=uvm_test_top.env0.agent1.*,_ALL_,UVM_HIGH,time,0`
+uvm_set_action	+uvm_set_action=<comp>,<id>,<severity>,<action> provides the equivalent of various uvm_report_object's set_report_*_action APIs. The special keyword, _ALL_, can be provided for both/either the id and/or severity arguments. The action can be UVM_NO_ACTION or a | separated list of the other UVM message actions (UVM_DISPLAY, UVM_LOG, UVM_COUNT, UVM_EXIT, UVM_CALL_HOOK, UVM_STOP). **Note**: As of UVM 1.1, this plusarg can only take a single UVM message action. You can not have a | seperated list of UVM message actions due to a bug in the UVM code.	`vsim testbench \ +uvm_set_action=uvm_test_top.env0.*,_ALL_,UVM_ERROR,UVM_NO_ACTION`
+uvm_set_severity	+uvm_set_severity=<comp>,<id>,<current severity>,<new severity> provides the equivalent of the various uvm_report_object's set_report_*_severity_override APIs. The special keyword, _ALL_, can be provided for both/either the id and/or current severity arguments.	`vsim testbench \ +uvm_set_severity=uvm_test_top.env0.*,BAD_CRC,UVM_ERROR,UVM_WARNING`
+uvm_set_inst_override +UVM_SET_INST_OVERRIDE +uvm_set_type_override +UVM_SET_TYPE_OVERRIDE	+uvm_set_inst_override=<req_type>,<override_type>,<full_inst_path> and +uvm_set_type_override=<req_type>,<override_type>[,<replace>] work like the name based overrides in the factory--factory.set_inst_override_by_name() and factory.set_type_override_by_name(). For uvm_set_type_override, the third argument is 0 or 1 (the default is 1 if this argument is left off); this argument specifies whether previous type overrides for the type should be replaced.	`vsim testbench \ +uvm_set_type_override=eth_packet,short_eth_packet`
+uvm_set_config_int +UVM_SET_CONFIG_INT +uvm_set_config_string +UVM_SET_CONFIG_STRING	+uvm_set_config_int=<comp>,<field>,<value> and +uvm_set_config_string=<comp>,<field>,<value> work like their procedural counterparts: set_config_int() and set_config_string(). For the value of int config settings, 'b (0b), 'o, 'd, 'h ('x or 0x) as the first two characters of the value are treated as base specifiers for interpreting the base of the number. Size specifiers are not used since SystemVerilog does not allow size specifiers in string to value conversions.	`vsim testbench \ +uvm_set_config_int=*,recording_detail,400`

User Defined Plusargs

In addition to the built-in plusargs that UVM provides, users can take advantage of the command line processor to add new plusargs. These new plusargs could be used in many ways including to pass in filenames for different initial memory images, change the number of iterations in a loop or configure the number of active masters/slaves in a switch or fabric environment.

Coding Guideline: Don't prefix user defined plusargs with "uvm_" or "UVM_". These prefixes are reserved by the UVM committee for future expansion of the built-in command line argument space. Do consider using a company and/or group prefix to prevent namespace overlap. For example, "MENT_" could be used by Mentor employees to denote user defined plusargs created by Mentor, A Siemens Business.

To get a value from a plusarg being set once on the command line code like this could be used:

```
uvm_cmdline_processor cmdline_proc = uvm_cmdline_processor::get_inst();
//get a string
string my_value = "default_value";
int rc = cmdline_proc.get_arg_value("+MENT_ABC=", my_value);
//Convert to an int
int my_int_value = my_value.atoi();
```

Notice that the value can be converted to an integer by using the standard built-in SV function atoi().

If you want to allow for the possibility of a plusarg being set multiple times, then you can use the get_arg_values() function. If the vsim command line looks like this:

```
vsim testbench +MENT_MY_ARG=500,NO +MENT_MY_ARG=200000,YES
```

the testbench could get those values by using the following code snippet.

```
uvm_cmdline_processor cmdline_proc = uvm_cmdline_processor::get_inst();
//get a string
string my_values[$];
int rc = cmdline_proc.get_arg_values("+MENT_MY_ARG=", my_values);
```

This would give me two entries in the my_values queue. They woudl be "500,NO" and "200000,YES". I could then write further code to split these strings using the uvm_split_string(string str, byte sep, ref string values[$]). This would look like this:

```
string split_values[$]
foreach (my_values[ii]) begin
  uvm_split_string(my_values[ii], ",", split_values);

  if (split_values[1] == "YES") begin
    ...
  end
end
```